International Journal of Information Security (2026) 25:75
https://doi.org/10.1007/s10207-026-01233-1

REGULAR CONTRIBUTION

Bridging remote attestation and secure software updates in embedded systems

Ahmad B. Usman[1] · **Zeeshan Afzal**[1] · **Mikael Asplund**[1]

Received: 5 September 2025 / Accepted: 14 February 2026 / Published online: 28 March 2026
© The Author(s) 2026

Abstract

Remote attestation enables centralized entities to assess the trustworthiness of remote devices. However, despite its utility, existing approaches often lack explicit support for secure software updates, which are essential for maintaining long-term security in embedded systems. This paper presents RASUES, a remote-attestation-based scheme specifically designed to integrate software update mechanisms into embedded systems. To achieve this, we extend the RATS (Remote ATtestation procedureS) RFC 9334 specification with an update procedure that accounts for both the expected state before and after the update. We implement the core functionality of RASUES on a TPM-based embedded hardware platform to demonstrate its feasibility. We evaluate the latency of the prototype and analyze its security properties, revealing that RASUES introduces minimal performance overhead while significantly enhancing security under defined assumptions. Furthermore, we conduct a comprehensive security analysis to identify potential threats and demonstrate how RASUES mitigates them. We also highlight areas in which the protocol must be complemented with additional security mechanisms, ensuring a transparent evaluation of RASUES's capabilities and limitations.

1 Introduction

Modern life increasingly depends on Internet of Things (IoT) devices, cyber-physical systems (CPS), and embedded systems to support a wide range of essential functions. A critical component of these systems is their software, which governs their behavior and functionality. However, this software is often vulnerable to cyber-attacks and may introduce security vulnerabilities if not properly protected.

The ability to update software in response to newly discovered vulnerabilities is an essential feature of any modern computing system. Regular updates are standard practice for mobile phones and computers, and well-managed cloud servers are typically updated as soon as a new patch is released. In contrast, embedded systems have traditionally followed a different approach. Often, software and firmware updates have been considered optional, placing the responsibility and effort to update on end-users, which carries potential security risks [57, 62]. However, this approach is set to change. Lawmakers and policy experts are pushing for stricter regulations, such as the European Cyber Resilience Act (which will apply to all products not covered by any other regulations) and the automotive ISO/SAE 21434 standard, both of which mandate the ability to perform security updates throughout a product's entire life cycle.

Remote attestation is an important security mechanism in the context of embedded systems that have been studied for many years [58, 61]. It enables a central entity (e.g., a cloud service) to verify that remote devices (e.g., IoT devices) have not been compromised and can thus be relied upon to perform their intended functionality within the system. In essence, remote attestation allows a verifier to validate the authenticity, configuration, and integrity of a remote device (the attester), thereby supporting security, trust, compliance, and correct system behavior.

While remote attestation offers significant security benefits, several challenges remain. These include its resilience to root-of-trust technology limitations [34] and its ability to protect against transient attacks [14]. Another limitation of many existing remote attestation mechanisms is that they do not explicitly account for the update process. Secure updates can influence the behavior of remote attestation, as the expected state of a device changes when an update is applied. In more complex environments, multiple valid versions of software or firmware may coexist, requiring the attestation verifier to

✉ Ahmad B. Usman
ahmad.usman@liu.se

[1] Department of Computer and Information Science, Linköping University, Linköping 58183, Sweden

track several acceptable states. Additionally, the update process itself may fail or be interrupted, highlighting the need for remote attestation to verify the success and integrity of the update.

Existing research emphasizes the importance of integrating secure updates with remote attestation [2, 39]. However, industry-standard initiatives for remote attestation, such as the Device Identifier Composition Engine (DICE) Attestation Architecture [55] by the Trusted Computing Group (TCG) and the Remote ATtestation ProcedureS (RATS) framework [7] developed by the Internet Engineering Task Force (IETF), do not specify how secure updates should be combined with the remote attestation process. This gap may result in ad-hoc implementations, which can inadvertently introduce design weaknesses and security vulnerabilities.

Some prior works have explored secure updates incorporating remote attestation [24]. For example, Plappert and Fuchs [41] focus on attestation-supported software updates within intra-vehicle communications. However, their research targets Electronic Control Units (ECUs) and adheres to the DICE specification, whereas our work addresses a more general remote attestation architecture aligned with the RATS specification and applicable to embedded systems. Other works in the literature have attempted to address challenges associated with secure updates using remote attestation [35, 42, 46]. However, these works either focus on cloud computing, sensor networks, and attestation attributes or do not rely on attestation specifications, such as the RATS RFC 9334 [7].

To address these challenges, we integrate remote attestation into the software update process to ensure the integrity and authenticity of updates, thereby enhancing the security and reliability of systems. We extend the RATS architecture by providing the connection to incorporate a secure update procedure into the system, mapping the required update steps onto the architectural components defined within RATS. Note that we do not replace the existing remote attestation procedure in RATS, but provide an integrated approach that supports both secure updates and remote attestation that adheres to the specification. While the extension is not inherently complex, it introduces essential mechanisms for maintaining an accurate record of the attester's expected state. To address potential update failures and defend against compromised hosts attempting to obstruct the update process, our protocol design integrates a watchdog timer mechanism that is tightly synchronized with the update workflow.

Building on our previous work [56], this paper offers three major advancements. First, we enhance the security analysis by not only verifying specific security properties in isolation but also systematically tracing how cyber-attack vectors can emerge and propagate across all three phases of the protocol. Second, we broaden our evaluation framework by benchmarking and comparing with prior work, situating our approach within the broader research landscape. Finally, we provide additional methodological and implementation details, including a dedicated background section, and provide a more detailed discussion of limitations and design choices. Our contributions to this paper are presented in a self-contained manner so that familiarity with the prior work is not required.

The proposed extension of the RATS specification, titled *Remote Attestation with Software Updates in Embedded Systems* (RASUES), integrates remote attestation, secure update mechanisms, and watchdog timers into a unified protocol. We validate its practicality by deploying RASUES on a Raspberry Pi platform with TPM 2.0 serving as the root-of-trust. The proof-of-concept evaluation quantifies the latency of each protocol phase and benchmarks these results against our pre-existing baseline framework. Finally, we define the essential security requirements, align the RASUES architecture with these criteria, and undertake a thorough security analysis. The contributions of this paper are as follows:

- Extension of an existing remote attestation specification (RFC-9334) to enable the incorporation of a secure update procedure into the system.
- Providing a proof-of-concept implementation of the RASUES scheme, including passport-model-based attestation.
- Conducting an evaluation and comparison analysis with the state-of-the-art and scientific literature.
- Performing a security analysis, demonstrating how RASUES mitigates identified threats and highlighting areas where it does not provide mitigation.

The remainder of our paper is structured as follows. In Section 2, we present an overview of remote attestation, secure update, and other concepts used in the paper. This section also explores the related work. In Section 3, we dive into the assumptions on the system's objectives and the threat model. Section 4 presents the high-level architecture of RASUES. In Section 5, we instantiate RASUES with a TPM. Section 6 presents the evaluation and results. Section 7 presents a detailed security analysis. Section 8 provides challenges and future work, and, finally, in Section 9 we draw our conclusions.

2 Background and related work

In this section, we provide an overview of software updates, remote attestation, and other technical concepts used in this paper. This section also discusses related works.

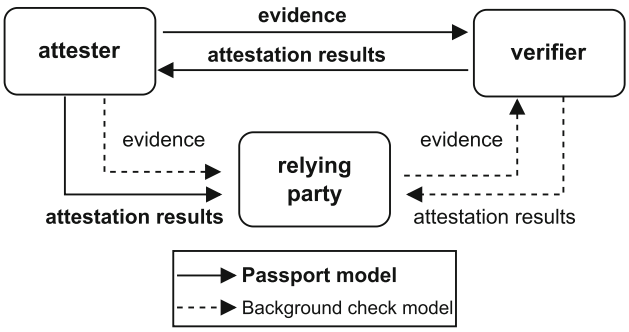


Fig. 1 Conceptual data flow of RATS.

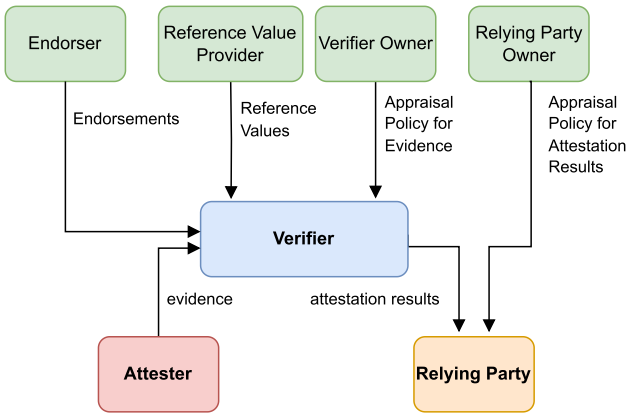


Fig. 2 Entities and roles in RATS.

2.1 Software updates

Software updates [6, 24, 46, 56] are important for maintaining the functionality, security, and performance of embedded systems. These updates often address vulnerabilities, introduce new features, and ensure compatibility with evolving technologies.

However, software updates in embedded systems also present challenges. If not adequately secured, software updates can become a major attack vector, enabling malicious actors to compromise the system by exploiting vulnerabilities in the update mechanism.

Consequently, securing software updates in embedded systems is critical to defending against evolving threats and ensuring long-term system trustworthiness.

2.2 Remote attestation

Remote attestation [17, 40, 57, 59, 60, 63] is a security mechanism in which a computer system or embedded device (commonly referred to as a prover, attester, or client) provides evidence or proof to a remote party (commonly referred to as a verifier or server), to confirm that it is operating in a trusted state. This mechanism is critical to ensure the authenticity and integrity of systems, particularly in embedded systems where trust is essential.

The remote attestation in RATS RFC 9334 specification [7] consists of three main entities and four other supportive entities communicating with each other and playing various roles, as shown in Figure 1. The RATS architecture has two patterns, the passport-model and the background check model. The passport model is named for its similarity to the process by which citizens acquire passports from their governments, whereas the background-check model is named for its resemblance to employer background screening processes. The two patterns are illustrated in Figure 2. We adopt the topological pattern of the passport-model from the RATS architecture.

Our preference for the passport-model over the background-check model is based on two main considerations. First, in the passport-model, verification occurs only when attestation is needed, that is, the verifier is contacted by the attester solely to obtain an attestation result, which can remain valid for a bounded period, depending on system requirements, or until an Authenticated Watchdog Timer (AWDT) event triggers re-attestation. In contrast, the background-check model requires contacting the verifier every time the relying party receives evidence. Therefore, using the background-check model makes the RASUES process dependent on the verifier's availability and introduces additional runtime overhead and latency, conditions that are particularly undesirable in IoT and embedded systems, where connectivity may be intermittent, and resources are constrained. Second, in the passport-model, the relying party processes only the attestation result, leaving the verifier only responsible for interpreting and validating the raw evidence. This separation simplifies the relying party's implementation, reduces its code footprint, and limits the attack surface.

Remote attestation can be broadly classified into three categories:

- Software-based attestation (e.g., SWATT [47], Real-SWATT [9, 32, 49]) relies entirely on software mechanisms to verify the integrity of a remote device.
- Hardware-based attestation (e.g., Sancus [22, 36]) uses dedicated hardware components such as TPMs or Physically Unclonable Functions (PUFs) to ensure system integrity.
- Hybrid-based attestation (e.g., HYDRA [21], VRASED [38]) combines both approaches to leverage their respective advantages.

In this paper, we focus on software-based attestation and hardware-assisted mechanisms using TPM as the root-of-trust. We use the terms attester and prover for consistency with the RATS specification [7] for the rest of this paper.

2.3 Authenticated watchdog timer (AWDT)

The Authenticated Watchdog Timer (AWDT) [1, 3, 27, 28, 63] provides a recovery mechanism for compromised remote devices. It operates by periodically counting down from a predefined time and, upon reaching zero, triggers a system reboot to restore the integrity of the device's firmware.

Several papers [1, 27, 28, 63] propose using AWDT to address security and reliability needs in systems like embedded systems, IoT devices [1], and critical computing environments.

Alternative drivers that are compatible with the secure update mechanism include ACPI, AMD FCH, Intel i6300esb, and Siemens SIMATIC IPC4x7E. However, these are general-purpose watchdog timers and may not provide the same security guarantees as AWDT or Authenticated Countdown Timer (ACT) [53].

2.4 Related work

Several works in the literature have investigated attestation mechanisms for update integrity [13, 32, 42]. To the best of our knowledge, this is the first work that extends a well-defined attestation standard with software updates and verifies its integrity , while simultaneously supporting hybrid attestation and explicitly considering device recoverability in the event of compromise, as shown in Table 1.

In the context of *embedded systems*, TrustLite framework [32] was introduced for providing security services, including attestation and secure software updates. It deploys a secure loader and an Execution-Aware Memory Protection on an FPGA for providing services. The ASSURED framework [2] was developed as an architecture tailored for secure software updates on large-scale deployments of low-end embedded devices. For verifying the updates, the framework utilizes hardware-based attestation mechanisms, specifically the ARM TrustZone security extensions. This also aligns with the PISTIS framework [25] while relying on only software-based attestation. Perito and Tsudik [41] introduced Proof of Secure Erasure (PoSE), a cryptographic technique enabling software updates on low-cost embedded systems without relying on secure hardware. Building on this concept, Karame et al. [29] proposed a more efficient update protocol to reduce PoSE's overhead. PURE [39] extends prior work to support secure updates in low-end embedded devices, ensuring that their post-update state remains malware-free.

In the field of *wireless sensor networks*, SCUBA [46] was proposed to support secure updates through attestation Although it includes a checksum-based mechanism to verify updates, SCUBA's reliance on precise timing may limit its applicability in embedded systems [10]. SUANT framework [29] was introduced by mixing PoSE with an all-or-nothing transform for update verifications.

In the domain of *automotive systems*, Plappert and Fuchs [41] propose attestation-assisted software updates for intra-vehicle communication, but their work does not address recovery from devices that have already been compromised. In contrast, our work presents a broader, high-level remote attestation architecture that explicitly explores how software updates and attestation mechanisms interact across key roles, including the verifier, reference value provider, and relying party.

In the context of *decentralized systems*, Rawat et al. [44] introduced a firmware attestation method for decentralized environments, though their approach targets ECUs without TPM support. CHAINIAC [36], relies on blockchain and consensus in its attestation process for verifying update integrity. In the area of *cloud computing* [44], attestation can be used for verifying the integrity of software updates with the support of TPM 1.2. Finally, the scheme LWSEE [11] was introduced for attestation and secure code updates with a focus on IoT devices that lack hardware-based capabilities.

Table 1 compares these studies. We categorized them based on their attestation type, application domain, software updates, and applicable standards. Among these studies, some employed hybrid-based attestation, and some used software-based attestation, while the remaining relied on hardware-based attestation. Regarding the application domain, some of the studies focused on embedded systems, followed by those addressing sensor wireless networks, IoT, and cloud systems. An alert symbol (▲) indicates that the corresponding study did not clearly specify the type of root-of-trust deployed, while the cross symbol (✖) indicates the absence of attestation specification.

3 System, threat model and properties

This section presents our system, threat model, and key security properties. In our system model, we identify four primary entities in the network, as illustrated below and presented in Figure 3.

- **Attester** - A partially untrusted device that may be compromised by an adversary. This can be any resource-constrained device, such as an IoT device, that must contain a trusted anchor like a Trusted Execution Environment (TEE) or a TPM.
- **Verifier** - A trusted entity that is responsible for verifying the current state of the attester.
- **Reference value provider** - A trusted entity responsible for supplying the expected state (reference value) to the verifier and delivering software updates. We use the

Table 1 Attestation schemes for verifying secure updates.

Attestation scheme	Attestation	Domain	Update	Recoverability	Standard
LWSEE [11]	Software	IoT	✔	✔	✖
SECURA [19]	Hardware	Railway	✔	✖	✔(RATS)
Plappert&Fuchs [41]	Hybrid	Automotive	✔	✖	✔(DICE)
PISTIS [25]	Software	Embedded	✔	✔	✖
CASU [13]	Hybrid	Embedded	✔	✖	✖
PURE [39]	Hybrid	Embedded	✔	✖	✖
Coughlin et al. [12]	Hybrid	Hardware	✔	✖	✖
ASSURED [4]	Hardware	Embedded	✔	✖	✔(TUF)
Feng et al. [22]	Hybrid	Embedded	✔	✖	✖
CHAINIAC [36]	Software	Debian	✔	✔	✖
Rama.& Lutfi. [44]	Software	Cloud	✔	✖	✔(TCG)
SUANT [29]	Software	Sensor	✔	✖	✖
Tan et al. [51]	Hardware	Sensor	✔	✔	✖
TrustLite [32]	Hybrid	Embedded	✔	✖	✖
PoSE-s [40]	Hybrid	Embedded	✔	✖	✖
SCUBA [46]	Software	Sensor	✔	✔	✖
RASUES	**Hybrid**	**Embedded**	✔	✔	**✔(RATS)**

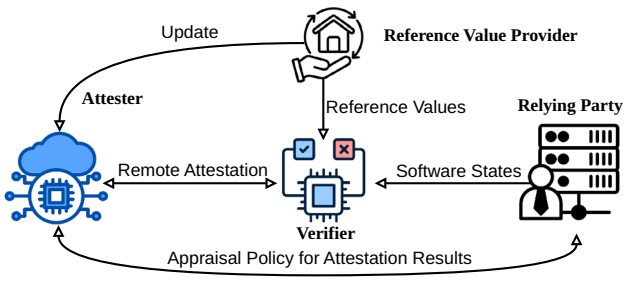


Fig. 3 System model.

term reference value provider instead of update agent to maintain consistency with the passport-model in RATS.

- **Relying party** - A trusted entity that validates the evidence of the attester based on the information and evidence appraised by the verifier.

3.1 Threat model

Types of Attackers: We define three types of attackers in our threat model, (i) network attacker ($\mathcal{A}_N$), (ii) software attacker ($\mathcal{A}_S$), and (iii) privileged attacker ($\mathcal{A}_P$) following the approach in [59]. $\mathcal{A}_N$ focuses on the communication channel, such as eavesdropping or intercepting the transmitted data, $\mathcal{A}_S$ focuses on the software, including the content of updates, and $\mathcal{A}_P$ focuses on the operating system.

Attacker Capabilities : We assume an adversary capable of compromising components of the software and firmware running on the attester device, in line with prior work [25, 41]. Specifically, the adversary can:

- Manipulate or boot the OS with malicious configuration.
- Intercept or block software updates.
- Prevent reboots initiated from a non-trusted environment.
- Launch remote code execution or privilege escalation attacks on non-trusted environments.

Despite these threats, we assume the existence of a secure attestation environment on the attester device (e.g., through a TEE or TPM), which remains intact and outside the adversary's control. We also assume the presence of a separate target environment, which is subject to monitoring and verification by the trusted attestation component.

Adversarial Goal The goal of the adversary is to compromise the confidentiality or integrity of the attester by having control over the communication channel or through unauthorized access (sniffing, blocking, modifying, replay, or spoofing).

Out-of-scope Threat Vectors: However, similar to attestation and secure software update literature [25, 41], we do not consider denial-of-service (DoS) attacks, or sophisticated physical attacks such as side-channel analysis, chip de-capsulation, or invasive probing. We also do not consider the time channel attacks on our root-of-trust (TPM), such as those demonstrated in [34].

3.2 Security properties

To meet the security requirements [40], we define four key security properties, denoted by (F_x^S).

- F_1^S (Updateability): Ensuring secure and reliable system updates.
- F_2^S (Recoverability): Ensuring system recovery in the event of an update failure.
- F_3^S (Freshness): Preventing replay attacks by ensuring recent communications.
- F_4^S (Integrity Assurance): Detecting malicious activities and preserving overall system security.

Further details on these are provided in Section 7. Beyond these, RASUES also supports additional security functionalities, including secure key generation, sealed storage, and encryption, leveraging the TPM as a hardware root-of-trust.

4 RASUES scheme

An overview of the RASUES scheme is presented in Figure 4. The protocol is structured into three main phases: (1) software update, (2) remote attestation, and (3) appraisal. The primary entities participating in the scheme are the attester, verifier, reference value provider, and relying party, which are defined in detail in 3:

The gray-colored components are derived from the Challenge Response-based Remote Attestation (CHARRA) mechanism [49] that follows the RATS RFC 9334 architecture [7]. We adopt the topological pattern of the passport-model from the RATS architecture, as illustrated earlier in Figure 2, and implement our scheme on top of the CHARRA [49]. Thus, RASUES extends CHARRA's challenge-response attestation mechanism by integrating software update verification and state tracking. A description of each phase is provided below.

4.1 Phase 1: software updates

The initial phase of the sequence diagram in Figure 4 illustrates the software update process. During this phase, mutual authentication is established among all participating entities using Transport Layer Security (TLS), specifically TLS 1.2, optimized for embedded systems through the use of mbedTLS [17]. Public and private key pairs are generated using elliptic curve cryptography (ECC), which supports secure key exchange and digital signatures to ensure confidentiality and authenticity in subsequent communications.

Notably, TLS communication is not established between the target environment and the attestation environment, since this is performed either within the CPU itself (in the case of a TEE) or on a hardware bus (assuming a TPM unit).

The reference value provider maintains updated information, such as reference values, to ensure that the verifier and attester have access to the latest valid system states. The attester is the designated device that receives software updates and verifies their authenticity using digital signatures and public key cryptography to confirm they originate from the reference value provider. The relying party is responsible for initiating the AWDT, while the attestation environment defines a timeout to monitor the update process. Since the duration of a software update may exceed typical AWDT timer settings, the AWDT remains active both during and after the update. To avoid unintended system resets, the relying party periodically refreshes the AWDT timeout, keeping it synchronized with the expected update duration. Finally, the verifier obtains the expected software state information from the reference value provider and updates its whitelist to reflect valid configurations for remote attestation.

4.2 Phase 2: remote attestation

In the second phase, the verifier initiates a remote attestation request to the attester. In response, the attester gathers relevant evidence, including firmware information, and generates an attestation report. This report is then signed using the attester's private key (ECC_{priv}) and transmitted to the verifier along with the corresponding public key (ECC_{pub}).

Upon receiving the response, the verifier undertakes a series of verification steps. It authenticates the digital signature using the attester's public key, checks the evidence against the expected state, and compares it to a regularly updated whitelist derived from reference values supplied by the reference value provider. If all verification steps are successful, the attestation is deemed valid; otherwise, it is rejected. To ensure freshness and protect against replay attacks, a unique nonce is generated for each attestation session.

4.3 Phase 3: appraisal

The final appraisal phase (as shown in Figure 4) evaluates the attestation evidence against the expected state, following verification by the verifier. This step is carried out by the relying party, which evaluates the received evidence against a predefined appraisal policy, analogous to the passport-model verification process described in RATS [7].

In addition, the attester operates in conjunction with an external AWDT.

If the attester fails to provide fresh attestation results within a predefined time set by the attestation environment, the AWDT triggers a system reboot, restoring the attester device to a known healthy and secure state. The relying party

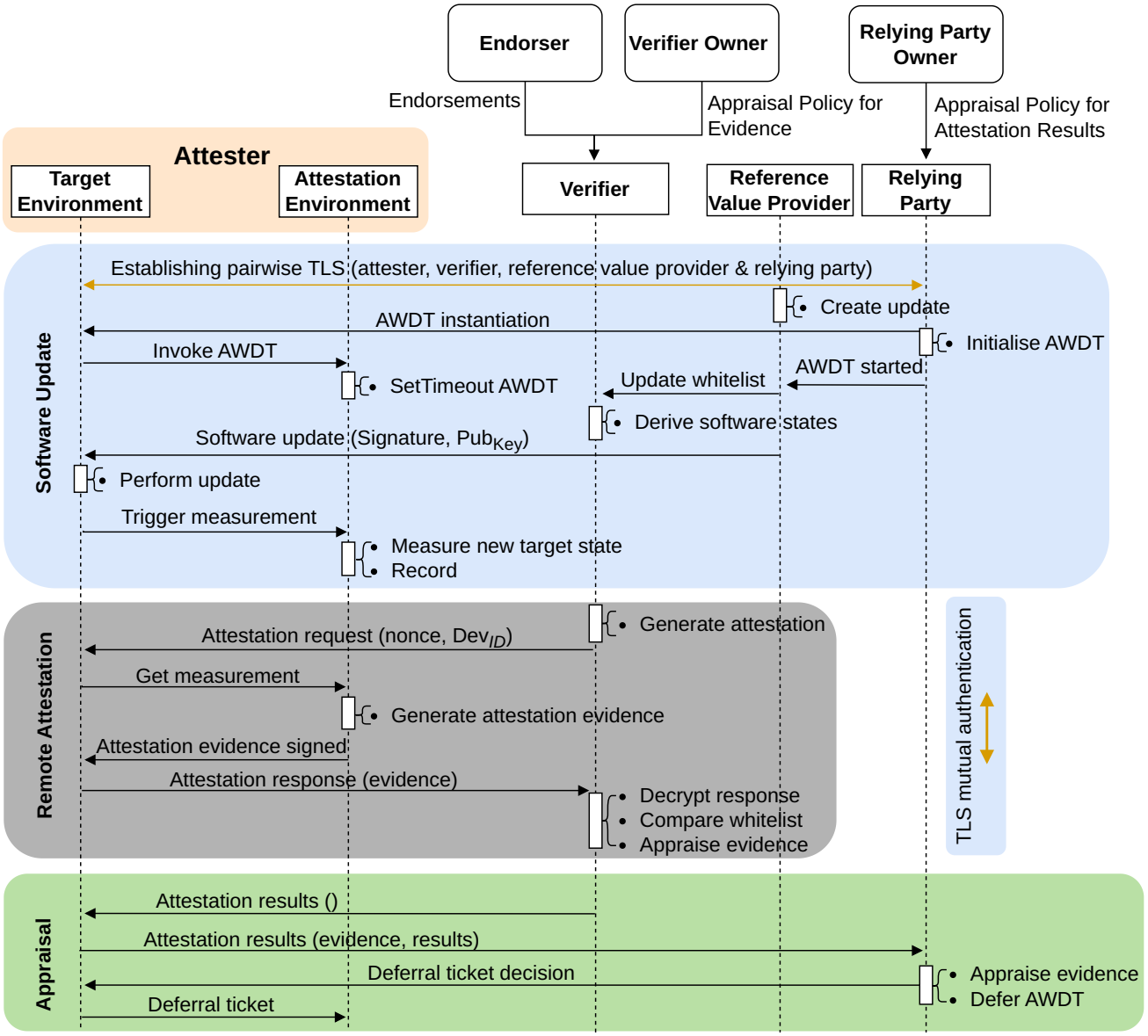


Fig. 4 RASUES overview: software update (phase 1), remote attestation (phase 2), appraisal (phase 3).

can continuously defer the AWDT as long as the attester operates correctly, indicating that no violations or anomalies are detected within its internal state. This approach aligns with the challenge-response protocol used to maintain AWDT integrity, as detailed in [28].

5 Implementation

In this section, we demonstrate the practical applicability of RASUES using a TPM as a root-of-trust. Our extension to CHARRA is centered on allowing incorporation of secure update capabilities. To achieve this, we adopt the SWUpdate framework as the update agent within the RASUES architecture. Additionally, we integrate the Integrity Measurement Architecture (IMA), which is a security subsystem available in modern Linux kernels, into the attestation environment to enable automatic integrity measurements. Consequently, this implementation of RASUES introduces stricter requirements for the attester compared to the more generic RASUES model, as it ideally treats even the operating system as outside the root-of-trust.

5.1 Trusted platform module

Our solution relies on establishing a root-of-trust within the attester device, which may otherwise be susceptible to compromise. A TPM is one such technology capable of

providing this root-of-trust, though alternative approaches include TEEs and specialized solutions tailored for resource-constrained devices [16, 39]. The TPM's primary role is to serve as a secure foundation for cryptographic operations, securely store sensitive data, and facilitate the process of verification and attestation.

The TPM maintains a set of Platform Configuration Registers (PCRs) for supporting attestation and the integrity measurement, which store integrity measurements representing the expected state of various system components during runtime.

These PCR values are standardized by the TCG [52] and are conventionally assigned to different system layers, spanning from early operating system boot phases to application-level components, as illustrated in Figure 5.

In our implementation, we leverage the Unified Extensible Firmware Interface (UEFI), which populates PCR2, PCR3, and PCR4, along with the IMA, which contributes measurements to PCR10.

The selected PCR indexes are indicated in Figure 5 with arrows showing how they are populated with usage information. The TPM also retains an Endorsement Key (EK), a cryptographic identifier unique to the TPM. This key is responsible for signing the measurements stored in the PCRs.

5.2 Software update creation

The software update mechanism utilizes the open-source SWUpdate framework[1], which serves as an update agent specifically designed for embedded systems. SWUpdate supports two main update strategies: single copy (non-A/B) and dual copy (A/B). The single-copy strategy follows an asymmetric update paradigm, where a standalone image is deployed. Although this approach is simpler, it requires the attester to reboot after each update and lacks built-in recovery mechanisms in case of failure. In contrast, the dual-copy strategy adopts a symmetric update paradigm, offering enhanced robustness by maintaining two system partitions, one active and one inactive, allowing safe fallback in the event of an update failure. However, this method requires double the storage capacity. For our implementation, we adopt the dual-copy strategy, as it ensures system recoverability in scenarios where the update process is interrupted or fails.

During the update generation process, the reference value provider creates the update package using SWUpdate, which produces a ready-to-deploy image based on the sw-description file, which allows us to define the components to be updated and their corresponding metadata. An .swu file is generated from the sw-description after successfully creating the update using the SWUpdate framework. Although updates can be automatically generated using tools like Yocto [45], our implementation manually creates the .swu file to ensure precise control over which system components are updated.

5.3 Managing and measuring expected state

The reference value provider creates the updates (cf. Section 3) and sends them to the attester. Upon receiving an update, the new expected state is derived and stored in the verifier's whitelist. This software update must then be reflected in a PCR of the TPM by performing the integrity measurement in the attestation environment.

Since TPM PCRs are append-only, the expected state of an attester is not modifiable directly. Instead, updates occur through the PCR Extend function, which appends new measurements by hashing the current value of the corresponding PCR with the new data, as formalized below in equation 1.

$$expected_state := H_{hashAlg}(Digest_{old}||Data_{new}) \quad (1)$$

where the

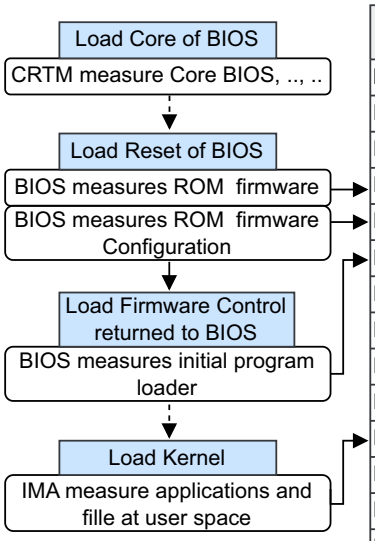
- $expected_state$ is the new valid digest of the PCR to be updated
- $H_{hashAlg}$ is a type of hash function such as SHA1, SHA256 or SHA384
- $Digest_old$ is the current state before update and
- $Data_new$ is the newly extended the data.

It is important to note that the update process is inherently non-reversible, which prevents the update sequence from being forged; that is, the expected state resulting from applying the update x followed by y is not equal to applying y followed by x: $expected_state(x, y) \neq expected_state(y, x)$.

Due to the irreversible nature of PCR updates, managing and tracking the expected state becomes increasingly complex, particularly in the context of frequent or critical system changes. Also, every time a new component is introduced or an existing one is modified, the verifier must record and maintain a history of all prior modifications, including software updates, in its whitelist.

On the attester side, software updates must be accurately reflected in the PCR values. To achieve this, we enable the IMA, a component integrated into the Linux kernel that automatically performs integrity measurements. IMA sequentially assesses individual OS files and applications to generate integrity metadata. To facilitate the automatic detection of modifications, we mount the file system with support for version tracking by enabling the i_version mount option. This ensures that each software update triggers a corresponding re-measurement of the software state and an update of the relevant PCR values.

[1] https://sbabic.github.io/swupdate



Expected states	
Index	**Usage**
PCR 0	BIOS, CRTM, Host Platform extension, ..
PCR 1	BIOS configuration
PCR 2	UEFI driver code and applications
PCR 3	UEFI configuration
PCR 4	UEFI boot manager code
PCR 5	Boot manager code configuration and data
PCR 6	Host platform manufacturer specific
PCR 7	Secure boot policy
PCR 8-9	Defined for use by the Static OS
PCR 10	IMA measurement
PCR 11-15	Defined for use by the Static OS
PCR 16	Debug
PCR 23	Application support

Fig. 5 PCR indexes and their relations to UEFI and IMA.

Table 2 provides an excerpt of the IMA measurement log, demonstrating how integrity measurements are recorded and associated with PCR values.

5.4 Attestation and watchdog timer

For the attestation phase, we employ CHARRA [49] in conjunction with TPM 2.0. We extend CHARRA to (i) implement the passport-model as defined in the RATS architecture, (ii) utilize the attestation mechanism to verify the integrity of the software update process, and (iii) evaluate and compare its performance as a baseline against the RASUES framework.

Currently, our prototype implementation of RASUES does not support the AWDT, as the TPM module used (Infineon SLB 9670) does not provide this functionality. However, a similar feature known as the Authenticated Countdown Timer (ACT) [53] has been introduced in TPM2.0 starting with revision 1.59. This feature can be leveraged to approximate and evaluate AWDT functionalities within our attestation framework.

5.5 Implementation platform

For the implementation, we use a Raspberry Pi 4 Model B equipped with a 64-bit ARM Cortex-A72 processor and 1GB of RAM. To enable TPM-based security functionalities, we attach an OPTIGA TPM SLB 9670 evaluation kit to the Raspberry Pi through its pin header using the Serial Peripheral Interface (SPI). The feasibility of this customization can also be seen in [30].

To facilitate communication with the TPM 2.0 module, we use Enhanced System API (ESAPI) [54] from tpm2-tools, based on tpm2-tss. Furthermore, for the asynchronous exchange of messages, we employ Constrained Application Protocol (CoAP) [48], which enables efficient asynchronous messaging for constrained embedded systems. SSL/TLS and DTLS APIs for secure communication are provided by mbed TLS. Finally, we use Constrained Binary Object Representation (CBOR) [8] for wire-encoding.

As mentioned before, the attestation environment is also equipped with an AWDT that resets and reboots the device if it fails to comply with the appraisal policy within a predetermined period. The experimental setup is illustrated in Figure 6. The upper part shows how the entities communicate, while the lower part details interactions between the

Table 2 Excerpt of IMA log.

PCR	template-hash	template	filedata-hash	filename-hint
10	fa4e17e78c0f4157d57c9d5ceaa1b39...	ima-ng	sha1:3b6f4ea7c9d766c5e65e3eb...	boot aggregate
10	a68468be32dff6deb0f47c9d4918238...	ima-ng	sha256:66d7c9d7a939af7c9de942d...	/init
10	fa4e17e78c0f4157d57c9d5ceaa1b39...	ima-ng	sha256:22db7117c9d46eaa213d31...	/usr/lib64/ld-2.16.so
10	b2344ac45817624290107c9db2be8e1...	ima-ng	sha256:397a5a97c9d9bb91a2c1335...	/etc/ld.so.cache
…	…	…	…	…

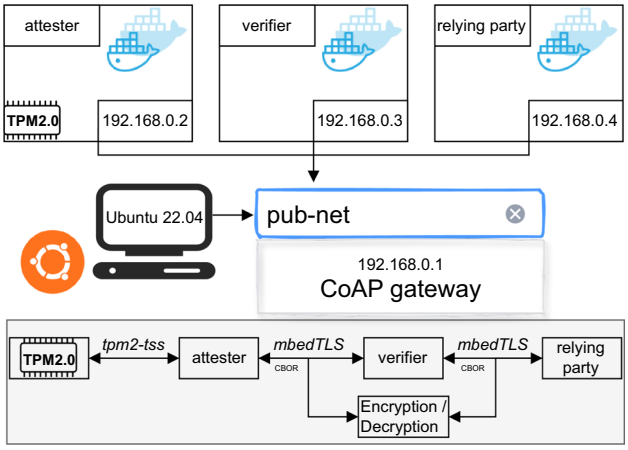


Fig. 6 Experimental setup through simulation.

TPM, tpm2-tss, TLS, and CBOR components. The specific hardware components used in the setup are further depicted in Appendix A and Figure 10.

6 Evaluation

This section evaluates the feasibility of RASUES in embedded systems by measuring its performance and comparing the results with other state-of-the-art solutions.

6.1 Performance evaluation

The evaluation is performed using one Raspberry Pi acting as the attester and a laptop containing virtualized instances of the other entities (verifier, reference value provider, and the relying party). The laptop was equipped with a 5-core Intel Core i5-6200U CPU running at 2.30 GHz, along with 8.0 GiB of RAM. The system operated on a 64-bit architecture and was running Ubuntu 22.04 LTS with the Linux 6.2 kernel.

The evaluation focuses on assessing the performance of RASUES and comparing its internal operational overhead with that of CHARRA [49]. Notably, the evaluation scenario considers a software update that does not require rebooting the attester. This allows the attestation environment to measure the attester's state in real-time. Throughout the update process, changes are tracked via the Platform Configuration Registers (PCRs) to ensure the expected state is accurately maintained (cf. Equation 1).

Initially, we evaluated the interaction of RASUES entities using the passport-model (cf. Figure 4) across 20 iterations, measuring execution time for the following entities:

- Verifier: Sending and verifying requests.
- Attester: Receiving and handling requests properly.
- Attester: Performing integrity measurement and forwarding results to the relying party.
- Relying party: Validating the attester's result.

Figure 8 presents a box plot demonstrating execution time. The x-axis denotes the specific iteration during execution, while the y-axis represents the corresponding execution time. The attester side incurs the longest execution time, reaching 415 milliseconds on average and 11 milliseconds to confirm the recipient of a request. In comparison, the verifier averages 196 milliseconds, and the relying party 252 milliseconds on average. The attester's extended execution time is attributed to the attestation environment.

Focusing on the attester, verifier, and relying party, we further extract the (i) proportion of CPU and memory consumed by the entity, (ii) data volume read and written between entities during the attestation, and (iii) the data transmitted and received during the attestation process via network interface.

Among the three entities, the attester exhibits the highest CPU usage (up to 5%), due to integrity measurements and TPM 2.0 interactions, followed by the verifier and the relying party, which demonstrate lower CPU usage with 2.5% and 1%, respectively. Furthermore, the attester consumes the most memory (23.04MB), exceeding the verifier (2.72MB) and reference value provider (2.55MB). This outcome is consistent with expectations, given the involvement of integrity measurement procedures and the supplementary computations associated with the TPM 2.0 of the attester in the attestation environment.

Data read-write patterns show that the attester reads more data than it writes, whereas the reference value provider exhibits the opposite behavior. Conversely, the verifier simultaneously performs both reading and writing operations. This stems from the verifier's involvement in rigorous actions on both the attester and its own side in the remote attestation phase.

Lastly, we consider the data transmitted and received through the pub-net interface at the CoAP gateway. We discovered that the attester transmits more data than it receives, while the verifier primarily receives data and waits for attestation responses. This distinction arises due to the fact that the verifier is in a state of waiting to receive an attestation response from the attester upon initializing the attestation request.

6.2 Comparison with CHARRA

To further evaluate the performance, we performed periodic attestation and compared the average latency of key RASUES functions with the baseline CHARRA, which serves as the baseline. This comparison was performed for both the verifier and attester, as illustrated in Figure 7. The dotted bars are

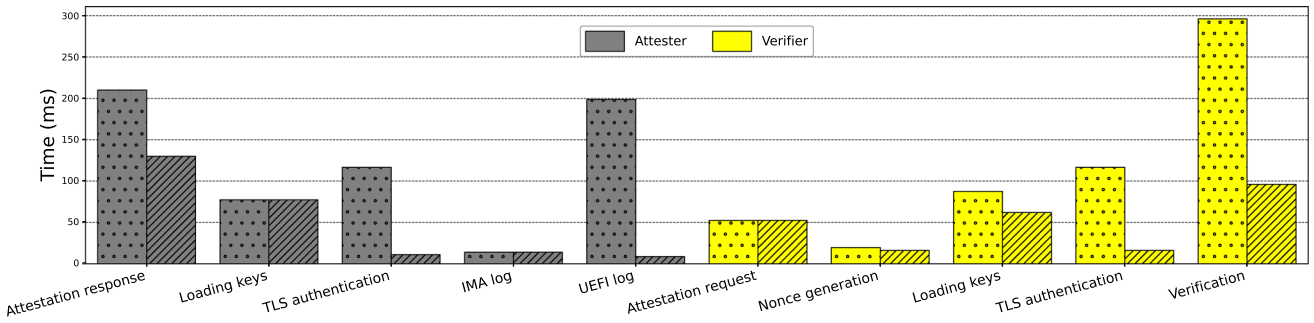


Fig. 7 Attestation Performance: Attester vs Verifier of RASUES functions (dotted bars) against the baseline (dashed bars)

Table 3 Description of attestation operation.

Operation	Description of operation (time to)
Nonce generation	Generate a nonce by verifier
Attestation request	Initiate attestation by verifier
Loading key	Implementing the generated secret keys
TLS authentication	TLS handshake and mutual authentication
IMA log measurement	IMA integrity measurement by attester
UEFI log measurement	PCRs integrity measurement by attester
Verification	Verify and validate the attester's state
Attestation response	Response to the attestation request by attester

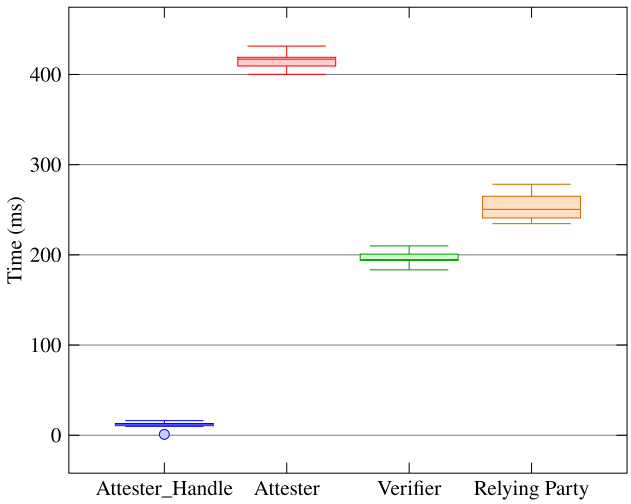


Fig. 8 Execution time of RASUES after 20 iterations.

RASUES functions, while the dashed bars are the baseline. The evaluated functions include nonce generation (to ensure freshness), initiation of the attestation request, and TLS handshake delays for mutual authentication between the verifier and the attester. The remaining evaluated functionalities are outlined in Table 3.

The results indicate that the remote attestation process in RASUES incurs higher latency in comparison to CHARRA. This outcome is expected and is particularly pronounced during the integrity measurement phase, due to the involvement of both the IMA and UEFI. Furthermore, additional delays are observed in the verification stage and in the round-trip time for attestation responses. The increased latency in RASUES results from additional security measures, such as IMA integration and extended integrity measurements.

6.3 Comparison with RACD

We further compared RASUES with RACD [18] and traditional remote attestation to assess the performance of both attestation and verification execution times. RACD's attestation time closely aligns with state-of-the-art methods, averaging 223 milliseconds, while traditional remote attestation averages 218 milliseconds, as shown in Figure 9. In comparison with the proposed system, RASUES attestation time averages 418 milliseconds, primarily due to IMA logging every measurement event. Additionally, in the passport-model, after receiving attestation results, the attester must respond with their complete measurement log, adding further delay. Finally, RASUES introduces additional delay due to an extra function that transmits attestation results to the relying party. On the other hand, during verification, both RAUSES and RACD processes demonstrate longer duration compared to traditional remote attestation, averaging 197 and 178 milliseconds, respectively, in contrast to the 13 milliseconds of the traditional attestation.

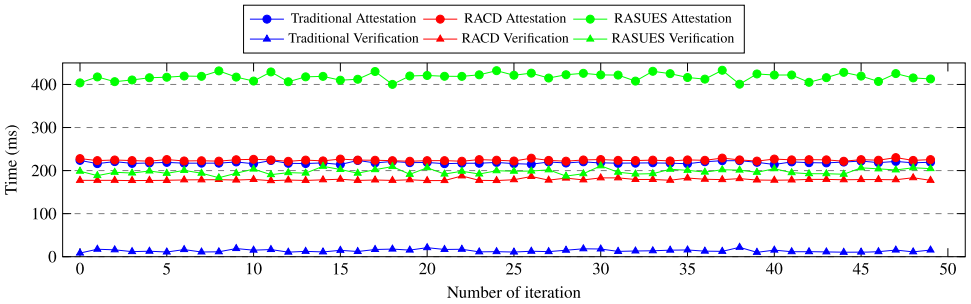


Fig. 9 Comparative analysis of RASUES, RACD [18], and traditional remote attestation methods, evaluating the performance based on both attestation and verification execution times over 50 iterations.

7 Security analysis

This section analyzes how RASUES satisfies the defined security properties of F_1^S(Updateability), F_2^S(Recoverability), F_3^S(Freshness), F_4^S(Integrity Assurance) under the considered attacker model comprising $\mathcal{A}_N$(Network attackers), $\mathcal{A}_S$(Software attackers), and $\mathcal{A}_P$(Privileged attackers).

RASUES is designed to support secure software updates while ensuring device recovery in the event of failures during the update process. This achieves the first two properties *Updateability* (F_1^S) and *Recoverability* (F_2^S). Furthermore, whenever the verifier communicates with the attester, a unique nonce is included in the attestation request to guarantee *Freshness* (F_3^S). We also incorporate *Integrity Assurance* (F_4^S) as a means to detect alterations and identify any unauthorized changes. However, despite these safeguards, security breaches may still occur, which motivates a systematic analysis of potential attack scenarios.

To this end, we conducted a structured security analysis based on the RASUES protocol sequence diagram in order to identify potential security threats and categorize them according to attacker capabilities and attack behavior. The identified threats include both passive attacks, such as sniffing and spoofing, and active attacks, such as blocking and modification. For clarity and completeness, these threats are further grouped into four attack scenarios: (i) sniffing, (ii) blocking, (iii) modification, and (iv) replay/spoofing. Replay and spoofing are considered jointly due to their close relationship. This classification is applied consistently across all three phases of the RASUES protocol using the following steps to enable a structured and comprehensive security analysis:

1. Threat Identification: Identify where threats arise.
2. Threat Description: Describe each threat and provide a representative example of how it could occur.
3. Impact Analysis: Analyzing the consequences of threats.
4. Attacker Association: Associate threats to specific attacker type (network, software, or privileged)
5. Security Property Mapping: Mapping to the affected security property.
6. Mitigation Analysis: Assessing whether the threat is mitigated by RASUES or requires additional countermeasures.

We now apply this approach to each phase of the RASUES protocol, analyzing the relevant attack scenarios and corresponding mitigations. A detailed analysis of all identified threats is summarized in the form of tables that cover the four attack scenarios (sniffing, blocking, modification, and replay/spoofing) across all protocol phases. This complete analysis together with the RASUES implementation, is available in our public repository [43]. Table 4 provides a high-level summary by mapping attacker types to the corresponding security properties and mitigation mechanisms.

7.1 Phase 1: software update

The software update phase consists of seven possible interactions between the RASUES entities within our considered scenarios. These scenarios could potentially result in various threats.

The adversary may sniff or block the communication channel between different entities: the relying party and the attester, the target environment and the attestation environment, and between the reference value provider, verifier, and the attester. This can lead to several consequences. A successful attack could compromise confidentiality and availability. For instance, the attacker can know when the AWDT is invoked/started and/or block it. More importantly, the attacker can learn the content of the update and the derived expected state and/or block it.

This threat can be mitigated by establishing secure communication using, e.g., TLS 1.2, ensuring trust through ECC-based cryptographic key exchange, and ensuring only authorized access is granted. TLS 1.2 was used for its mature support in the mbedTLS library and widespread use in resource-constrained and trusted embedded devices. The proposed approach is not dependent on a specific TLS version. TLS is also not established between the target environment and the attestation environments, as we assume secure communication via a hardware TPM bus or an internal TEE. At the same time, certain types of active sniffing can still com-

Table 4 Mapping attackers with security properties.

Attack type	Property
$\mathcal{A}_N$(Network)	F_4^S(Integrity Assurance), F_3^S(Freshness)
$\mathcal{A}_S$(Software)	F_1^S (Updateability), F_4^S(Integrity Assurance)
$\mathcal{A}_P$(Privileged)	F_2^S (Recoverability), F_4^S(Integrity Assurance)

promise the confidentiality of the communicationdespite the presence of TLS (since the attester might be compromised). In such cases, an attacker may disrupt the communication channel, for example, by launching a DoS attack. This disruption could prevent the attester from receiving the intended software update. This is a serious issue, especially within safety-critical systems, but we do not consider this type of attack in the present paper, in line with the assumptions made in [41].

An attacker might also attempt to change the attester's state by installing malicious updates. When this occurs, the attestation will fail, causing the device to reboot into a trusted state, allowing the update process to continue safely afterward. Additionally, there is a risk of a malicious update being sent to a legitimate attester. To mitigate this, the reference value provider uses public key cryptography to sign updates, ensuring the attester only applies updates that have been properly verified.

7.2 Phase 2: remote attestation

The remote attestation phase involves four interactions between entities, potentially leading to more threat scenarios.

The adversary may attempt to sniff, block, or modify the two-way communication channel between the verifier and the attester, as well as the two-way communication channel between the target environment and the attestation environment. If successful, this attack compromises attestation requests and responses, including integrity measurements, undermining protocol trustworthiness. For instance, an attacker may attempt to reuse messages to perform a replay attack. To mitigate this attack, RASUES derives attestation integrity from CHARRA, ensuring freshness and providing basic remote attestation security.

7.3 Phase 3: appraisal

The appraisal phase, like phase 2, consists of four interactions and results in the identification of additional threats.

The adversary may attempt to sniff, block, or modify the communication channel. Specifically, between the verifier delivering the attestation results to the attester, or evidence, and the results sent from the attester to the relying party, fulfilling the passport model. And finally, between the relying party and the attester, and the target environment to the attestation environment.

If successful, such an attack could deceive the relying party into accepting falsified evidence, potentially resulting in incorrect operational decisions. To counter this, the attestation evidence is cryptographically signed by the verifier, ensuring that only the designated attester, capable of decrypting the request with its private key, can respond. The verifier then confirms the authenticity of the response. Furthermore, the attestation environment conducts integrity measurements of the device's current state, allowing it to detect any deviations from the expected state.

An attack targeting the communication channel between the relying party and the attester could enable an adversary to inject a forged decision or deferral ticket. This could cause the relying party to trust the attester unconditionally or to accept an outdated, but previously valid, state as legitimate, allowing a compromised device to continue functioning undetected.

Possible mitigation for this attack includes using the watchdog timer with a nonce-based challenge-response mechanism [20].

Other security aspects: Beyond the main functions, RASUES supports additional security functionality that can be achieved through the derived tools. Firstly, the usage of TPM 2.0 automatically enhances the security posture and generates secure cryptographic keys. This can help mitigate several attacks, such as physical tampering [5], or help avoid Time-of-Check-to-Time-of-Use (TOCTOU) attack [31].

Atomicity can also be achieved with SWUpdate. In phase 1, the update installation can be unsuccessful due to a power failure, network losses, or, more importantly, a cyber attack. RASUES leverages a dual-copy (A/B) approach to ensure that updates are fully installed before activation. The copy 'A' will be an active and running update, and the copy 'B' is inactive but a stand-by copy, to guarantee the availability of a working package update. The attestation environment must exchange information with the reference value provider to detect an update interruption in order to activate the stand-by copy. In phase 2, the attestation process is entirely carried out as a single and coherent process, successfully realizing the expected state without interruption. The absence of this functionality can leave room for roving malware [9], where it partially executes and erases itself to reappear later before the end of the attestation process.

It is worth noting that this paper does not prevent against roll-back attacks [19]. However, to mitigate these attacks, we adopt a dual-copy strategy similar to that employed in the SWUpdate framework (cf. §5.2).

In our prototype implementation of RASUES, IMA is considered part of the attestation environment. However, IMA itself could potentially be compromised by the attacker who may gain access to kernel-level privileges. This would break the security of our approach. Thus, it is an interesting venue for future work on how to implement non-trivial integrity measurements without relying on the security of the kernel.

8 Outlook

The aim of this paper is to enhance the security and reliability of embedded systems by integrating remote attestation into the update mechanisms in accordance with the reliable attestation specification. However, security in computing systems is an ongoing challenge, as no system can offer absolute protection. New threats and vulnerabilities continuously emerge, necessitating regular software updates to maintain the integrity and functionality of embedded devices. Timely software updates in embedded systems are important to maintain the security and functionality of these devices. It is thus crucial to ensure that systems are equipped to address these challenges.

The results of this work suggest that RASUES can effectively extend traditional remote attestation mechanisms to support software updates in embedded systems. Our approach integrates several important elements, including the TPM-based PCR monitoring and secure communication via TLS, and the integrity measurement using IMA to the attester in the attestation environment. In doing so, RASUES formulates four security functionalities (cf. 3.2) to mitigate several common attacks to verify the update integrity and ensure that only authorized modifications are applied to the attester's state.

While RASUES mitigates a broad range of attacks, certain types of attacks, including time channel attacks, remain out of scope, as mentioned in Section 3. Furthermore, although we did not observe this behavior empirically, a compromised attester could in theory repeatedly force reboots, which in turn could potentially lead to a deadlock or persistent operational instability.

Moreover, our measurement approach, based on the IMA log, contains deterministic hashes. Since the log uses a template hash (cf. Table 2), an attacker could extract specific details about a particular binary [18]. Even if RASUES were to use SHA-512 instead of the suggested SHA-1, this issue would persist. Additionally, transmitting the entire measurement log for verification raises privacy issues. The granularity of the data could inadvertently expose sensitive details regarding the attester's configuration and deployed update. Addressing this limitation may require the integration of privacy-preserving techniques, such as zero-knowledge attestation [15, 18] that limit the sensitive information shared with the verifier. In the context of these privacy concerns, prior work such as RACD [18]) has used Schnorr-based non-interactive zero-knowledge (NIZK) proofs to enable selective disclosure of measurement entries while preserving verifier trust. Although Schnorr is a signature scheme rather than a hash, its structure supports efficient NIZK proofs (cf. RFC 8235 [26]), which can be considered as a possible mechanism for enhancing privacy in RASUES.

Finally, an attacker could manually invoke certain update-related commands, then provide a valid signature to the verifier after submitting a software update. This could enable a TOCTOU attack, where the attacker verifies the PCR values after the update, and then modifies the result before the PCR update, ensuring it matches the expected state of the attester. While RASUES currently does not consider TOCTOU attacks, the design could integrate an Integrity Key (IK) mechanism tied to PCR measurements, similar to [31]. In such a design, the attestation key is cryptographically bound to the device's software state at the time of measurement. Any subsequent modification of the software before PCR updates would invalidate the key, thereby preventing TOCTOU exploitation.

Future work may also involve formal protocol verification to reveal further insights into potential vulnerabilities and assist in developing a sound defense against related attacks. Evaluating the scheme under stronger adversarial models is another avenue for future work.

9 Conclusions

This paper presents an approach for performing software updates in embedded systems using remote attestation. The work extends the existing RATS remote attestation architecture, leveraging TPM 2.0 as a root-of-trust. Specifically, we build upon CHARRA, which provides a basic attestation mechanism, to enable the incorporation of a secure update procedure into the system, while adhering to the specification. To evaluate the feasibility and effectiveness of our approach, we conduct proof-of-concept experiments, examining deployment practicality, update frequency, and remote attestation performance. Additionally, we compare RASUES with existing state-of-the-art solutions to assess its performance and potential trade-offs. Furthermore, we perform a security analysis of our protocol to identify any possible weaknesses. We demonstrate that RASUES can mitigate certain security threats while also acknowledging its limitations. These insights contribute to the ongoing development of secure software update mechanisms for embedded systems.

Appendix A: Supplementary material

Appendix A.1: Update Generation using SWUpdate

Example generating update with SWUpdate shown in List 1. The generated update may involve updating a portion of the attester, as shown in List 2 or updating the whole OS of the attester, as shown in List 3.

```
%CONTAINER_VER="1.0"
%PRODUCT_NAME="rasues-file-update"
%FILES="sw-description rasues
    OS_VERSION.txt update.sh"
%openssl dgst -sha256 -sign priv.pem
    sw-description > sw-description.
    sig
%for i in $FILES;do
%    echo $i;done | cpio -ov -H crc >
     ${PRODUCT_NAME}_${CONTAINER_VER}.
    swu
%
```

Listing 1 Generating software update.

```
software = { version = "0.1.0";
     raspberrypi4 ={
          hardware-compatibility: [ "
    1.0"]; stable:{
copyA : {
 filename: ({filename = "rasues";
     path = "/mnt/root/usr/bin/rasues"
    ;});
 filename: ({filename = "OS_VERSION";
       path = "/mnt/root/home/root/
    OS_VERSION";});
     scripts: ({filename = "update.sh"
    ;
          type = "shellscript";});};
copyB : {
 filename: ({filename = "rasues";
     path = "/mnt/root/usr/bin/rasues"
    ;});
 filename: ({filename = "OS_VERSION";
     path = "/mnt/root/home/root/
    OS_VERSION";});
     scripts: ({filename = "update.sh"
    ;
          type = "shellscript";});
};}}}
```

Listing 2 Updating portion of attester.

```
software = { version = "0.1.0";
     raspberrypi4 = {
          hardware-compatibility: [ "
    1.0"]; stable:{
copyA: {
     images: (
          {filename = "core-image-full-
    cmdline-raspberrypi4.ext4.gz";
          type = "raw";
          compressed = true;
          device = "/dev/mmcblk0p2";});
     scripts: ({filename = "emmcsetup.
    lua";
          type = "lua";});
     uboot: ({name = "rpipart"; value=
    "2";});};
copyB: {
     images: ({
          filename = "core-image-full-
    cmdline-raspberrypi4.ext4.gz";
          type = "raw";
          compressed = true;
          evice = "/dev/mmcblk0p3";});
     scripts: ({
          filename = "emmcsetup.lua";
          type = "lua";});
     uboot: ({name = "rpipart"; value=
    "3";});
};}}}
```

Listing 3 Updating full image of attester.

The produced update from SWUpdate can manually be measured in TPM as shown in List 4, while taking Equation 1 into consideration.

The output should be similar to the excerpts shown in List 5, and the reflection of the update must be identical. If violated, the verifier will report inconsistency, and thus, the violation will be detected during attestation. More details on attestation can be explored in the TPM-JS[2] simulation environment.

```
// Update with tpm2-tools using TLS
software_update=$(echo -n "rasues-
    file-update_1.0.swu" | openssl
    dgst -sha256 -binary | xxd -p -c
    32)
// Reflect Update by the reference
    value provider
tpm2_pcrextend 10:sha256=
    $software_update
```

Listing 4 Updating measurment in TPM.

```
//expected output should be identical
Expected: 0cb23c5af7fe9bbx...
    z35ed457d30e60fa
Actual   : 0cb23c5af7fe9bbx...
    z35ed457d30e60fa
OK
```

Listing 5 Measurement output.

[2] https://google.github.io/tpm-js/#pg_attestation

Appendix A.2: Evaluation tool

The evaluation tool consists of a Raspberry Pi 4B with a 64-bit ARM Cortex A72 processor and OPTIGA TPM SLB 9670 evaluation kit, as shown in Figure 10.

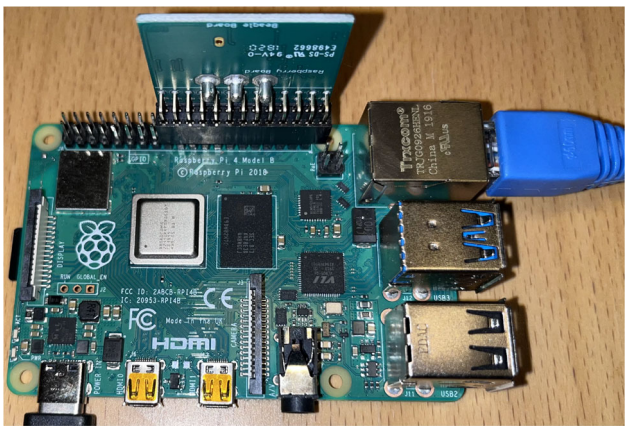


Fig. 10 Raspberry Pi 4B with a 64-bit ARM Cortex A72 processor and OPTIGA TPM SLB 9670 evaluation kit.

Acknowledgements This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

Author Contributions Ahmad B. Usman: Contributed by designing the protocol, experimental analysis, and writing the majority of the manuscript. Zeeshan Afzal: Provided support with conceptualization, methodology, review, and revision of the manuscript. Mikael Asplund: Provided support with conceptualization, methodology, review, and revision of the manuscript and secured funding.

Funding Open access funding provided by Linköping University.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Al Atiiq, S., Gehrmann, C.: Regaining dominance in cider and lazarus. IEEE Access **12**, 124589–124603 (2024). https://doi.org/10.1109/ACCESS.2024.3454555
2. Asokan, N., Nyman, T., Rattanavipanon, N., Sadeghi, A.R., Tsudik, G.: Assured: Architecture for secure software update of realistic embedded devices. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. (2018). https://doi.org/10.1109/TCAD.2018.2858422
3. AWDT_2020, StefanT T, Clifford EB, Paul E, Dennis James M, Marcus P (2020) WO2020197805 -CRYPTOGRAPHICHARDWARE. https://patentscope.wipo.int/search/en/detail.jsf?docId=WO2020197805. Accessed 25 Feb 2026
4. Barbareschi, M., Battista, E., Mazzeo, A., Venkatesan, S.: Advancing wsn physical security adopting tpm-based architectures. In: Proceedings of the 2014 IEEE 15th International Conference on Information Reuse and Integration (IEEE IRI 2014), pp. 394–399 (2014). https://doi.org/10.1109/IRI.2014.7051916
5. Bellissimo, A., Burgess, J., Fu, K.: Secure software updates: Disappointments and new challenges. In: First USENIX Workshop on Hot Topics in Security (HotSec 06). USENIX Association, Vancouver, B.C. Canada (2006). https://www.usenix.org/conference/hotsec-06/secure-software-updates-disappointments-and-new-challenges
6. Birkholz, H., Thaler, D., Richardson, M., Smith, N., Pan, W.: Rfc 9334: Remote attestation procedures (rats) architecture (2023). 10.17487/RFC9334
7. Bormann, C., Hoffman, P.: Concise binary object representation (cbor). Tech. rep. (2013). https://datatracker.ietf.org/doc/html/rfc8949
8. Carpent, X., Rattanavipanon, N., Tsudik, G.: Remote attestation of iot devices via smarm: Shuffled measurements against roving malware. In: 2018 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), pp. 9–16 (2018). https://doi.org/10.1109/HST.2018.8383885
9. Castelluccia, C., Francillon, A., Perito, D., Soriente, C.: On the difficulty of software-based attestation of embedded devices. In: Proceedings of the 16th ACM Conference on Computer and Communications Security, CCS '09, p. 400–409. Association for Computing Machinery, New York, NY, USA (2009). https://doi.org/10.1145/1653662.1653711
10. Cecílio, J., de Sá, A.O., Jäger, G., Souto, A., Casimiro, A.: Lwsee: Lightweight secured software-based execution environment. Internet of Things **30**, 101513 (2025). https://doi.org/10.1016/j.iot.2025.101513
11. Coughlin, A., Cusack, G., Wampler, J., Keller, E., Wustrow, E.: Breaking the trust dependence on third party processes for reconfigurable secure hardware. In: Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, FPGA '19, p. 282–291. Association for Computing Machinery, New York, NY, USA (2019). DOI: 10.1145/3289602.3293895
12. De Oliveira Nunes, I., Jakkamsetti, S., Kim, Y., Tsudik, G.: Casu: Compromise avoidance via secure update for low-end embedded systems. In: Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design, ICCAD '22. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3508352.3549450
13. De Oliveira Nunes, I., Jakkamsetti, S., Rattanavipanon, N., Tsudik, G.: On the toctou problem in remote attestation. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS '21, p. 2921–2936. Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3460120.3484532

14. Debes, H.B., Dushku, E., Giannetsos, T., Marandi, A.: Zekra: Zero-knowledge control-flow attestation. In: Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security, ASIA CCS '23, p. 357–371. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3579856.3582833
15. Defrawy, K.M.E., Tsudik, G., Francillon, A., Perito, D.: Smart: Secure and minimal architecture for (establishing dynamic) root of trust. In: Network and Distributed System Security Symposium (2012). https://api.semanticscholar.org/CorpusID:909934
16. Dierks, T., Rescorla, E.: Rfc 5246: The transport layer security (tls) protocol version 1.2 (2008)
17. Eckel, M., George, D.R., Grohmann, B., Krauß, C.: Remote attestation with constrained disclosure. In: Proceedings of the 39th Annual Computer Security Applications Conference, ACSAC '23, p. 718–731. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3627106.3627118
18. Eckel, M., Gürgens, S.: Secura: Unified reference architecture for advanced security and trust in safety critical infrastructures. In: Proceedings of the 19th International Conference on Availability, Reliability and Security, ARES '24. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3664476.3664513
19. Eckel, M., Gutsche, T., Lauer, H., Rein, A.: A generic iot quantum-safe watchdog timer protocol. In: Proceedings of the 18th International Conference on Availability, Reliability and Security, ARES '23. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3600160.3605169
20. Eldefrawy, K., Rattanavipanon, N., Tsudik, G.: Hydra: hybrid design for remote attestation (using a formally verified microkernel). In: Proceedings of the 10th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WiSec '17, p. 99–110. Association for Computing Machinery, New York, NY, USA (2017). https://doi.org/10.1145/3098243.3098261
21. Feng, W., Qin, Y., Zhao, S., Liu, Z., Chu, X., Feng, D.: Secure code updates for smart embedded devices based on pufs. In: S. Capkun, S.S.M. Chow (eds.) Cryptology and Network Security, pp. 325–346. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-030-02641-7_15
22. Geden, M., Rasmussen, K.: Hardware-assisted remote runtime attestation for critical embedded systems. In: 2019 17th International Conference on Privacy, Security and Trust (PST), pp. 1–10 (2019). https://doi.org/10.1109/PST47121.2019.8949036
23. Ghosal, A., Halder, S., Conti, M.: Secure over-the-air software update for connected vehicles. Comput. Netw. **218**, 109394 (2022)
24. Grisafi, M., Ammar, M., Roveri, M., Crispo, B.: PISTIS: Trusted computing architecture for low-end embedded systems. In: 31st USENIX Security Symposium (USENIX Security 22), pp. 3843–3860. USENIX Association, Boston, MA (2022). https://www.usenix.org/conference/usenixsecurity22/presentation/grisafi
25. Hao, F.: Schnorr Non-interactive Zero-Knowledge Proof. RFC 8235 (2017). 10.17487/RFC8235
26. Huber, M., Hristozov, S., Ott, S., Sarafov, V., Peinado, M.: The lazarus effect: Healing compromised devices in the internet of small things. In: Proceedings of the 15th ACM Asia Conference on Computer and Communications Security, ASIA CCS '20, p. 6–19. Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3320269.3384723
27. Jäger, L., Lorych, D., Eckel, M.: A resilient network node for the industrial internet of things. In: Proceedings of the 17th International Conference on Availability, Reliability and Security, ARES '22. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3538969.3538989
28. Karame, G.O., Li, W.: Secure erasure and code update in legacy sensors. In: M. Conti, M. Schunter, I. Askoxylakis (eds.) Trust and Trustworthy Computing, pp. 283–299. Springer International Publishing, Cham (2015). https://doi.org/10.1007/978-3-319-22846-4_17
29. Khurshid, A., Raza, S.: Autocert: Automated toctou-secure digital certification for iot with combined authentication and assurance. Computers & Security **124**, 102952 (2023). https://doi.org/10.1016/j.cose.2022.102952
30. Khurshid, A., Raza, S.: Autocert: Automated toctou-secure digital certification for iot with combined authentication and assurance. Computers & Security **124**, 102952 (2023). https://doi.org/10.1016/j.cose.2022.102952https://www.sciencedirect.com/science/article/pii/S0167404822003443
31. Koeberl, P., Schulz, S., Sadeghi, A.R., Varadharajan, V.: Trustlite: A security architecture for tiny embedded devices. In: Proceedings of the Ninth European Conference on Computer Systems, EuroSys '14. Association for Computing Machinery, New York, NY, USA (2014). https://doi.org/10.1145/2592798.2592824
32. Kumar, S., Eugster, P., Santini, S.: Software-based remote network attestation. IEEE Trans. Dependable Secure Comput. **19**(5), 2920–2933 (2022). https://doi.org/10.1109/TDSC.2021.3077993
33. Moghimi, D., Sunar, B., Eisenbarth, T., Heninger, N.: TPM-FAIL: TPM meets timing and lattice attacks. In: 29th USENIX Security Symposium (USENIX Security 20), pp. 2057–2073. USENIX Association (2020). https://www.usenix.org/conference/usenixsecurity20/presentation/moghimi-tpm
34. Nauman, M., Alam, M., Zhang, X., Ali, T.: Remote attestation of attribute updates and information flows in a ucon system. In: Chen, L., Mitchell, C.J., Martin, A. (eds.) Trusted Computing, pp. 63–80. Springer, Berlin Heidelberg, Berlin, Heidelberg (2009)
35. Nikitin, K., Kokoris-Kogias, E., Jovanovic, P., Gailly, N., Gasser, L., Khoffi, I., Cappos, J., Ford, B.: CHAINIAC: Proactive Software-Update transparency via collectively signed skipchains and verified builds. In: 26th USENIX Security Symposium (USENIX Security 17), pp. 1271–1287. USENIX Association, Vancouver, BC (2017). https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/nikitin
36. Noorman, J., Agten, P., Daniels, W., Strackx, R., Van Herrewege, A., Huygens, C., Preneel, B., Verbauwhede, I., Piessens, F.: Sancus: Low-cost trustworthy extensible networked devices with a zero-software trusted computing base. In: Proceedings of the 22nd USENIX Conference on Security, SEC'13, p. 479–494. USENIX Association, USA (2013)
37. Nunes, I.D.O., Eldefrawy, K., Rattanavipanon, N., Steiner, M., Tsudik, G.: VRASED: A verified Hardware/Software Co-Design for remote attestation. In: 28th USENIX Security Symposium (USENIX Security 19), pp. 1429–1446. USENIX Association, Santa Clara, CA (2019). https://www.usenix.org/conference/usenixsecurity19/presentation/de-oliveira-nunes
38. de Oliveira Nunes, I., Eldefrawy, K., Rattanavipanon, N., Tsudik, G.: Pure: Using verified remote attestation to obtain proofs of update, reset and erasure in low-end embedded systems. In: 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), pp. 1–8 (2019). https://doi.org/10.1109/ICCAD45719.2019.8942118
39. Perito, D., Tsudik, G.: Secure code update for embedded devices via proofs of secure erasure. In: D. Gritzalis, B. Preneel, M. Theoharidou (eds.) Computer Security - ESORICS 2010, pp. 643–662. Springer, Berlin Heidelberg, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15497-3_39
40. Plappert, C., Fuchs, A.: Secure and lightweight ecu attestations for resilient over-the-air updates in connected vehicles. In: Proceedings of the 39th Annual Computer Security Applications Conference, ACSAC '23, p. 283–297. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3627106.3627202
41. Ramachandran, K., Lutfiyya, H.: A remote attestation infrastructure for verifying the application of software updates. In: 2017

IFIP/IEEE Symposium on Integrated Network and Service Management (IM), pp. 317–325 (2017). 10.23919/INM.2017.7987294
42. RASUES: https://github.com/abusm4n/rasues. Accessed: 2025–12-23
43. Rawat, A., Khodari, M., Asplund, M., Gurtov, A.: Decentralized firmware attestation for in-vehicle networks. ACM Trans. Cyber-Phys. Syst. 5(1) (2021). DOI: 10.1145/3418685
44. Salvador, O., Angolini, D.: Embedded Linux Development with Yocto Project. Packt Publishing Ltd (2014)
45. Seshadri, A., Luk, M., Perrig, A., van Doorn, L., Khosla, P.: Scuba: Secure code update by attestation in sensor networks. In: Proceedings of the 5th ACM Workshop on Wireless Security, WiSe '06, p. 85–94. Association for Computing Machinery, New York, NY, USA (2006). https://doi.org/10.1145/1161289.1161306
46. Seshadri, A., Perrig, A., van Doorn, L., Khosla, P.: Swatt: software-based attestation for embedded devices. In: IEEE Symposium on Security and Privacy, 2004. Proceedings. 2004, pp. 272–282 (2004). https://doi.org/10.1109/SECPRI.2004.1301329
47. Shelby, Z., Hartke, K., Bormann, C.: The constrained application protocol (coap). Tech. rep. (2014). https://datatracker.ietf.org/doc/html/rfc7252
48. SIT, F.: Charra: Challenge-response based remote attestation with tpm 2.0. Fraunhofer-SIT (2019). https://github.com/Fraunhofer-SIT/charra
49. Surminski, S., Niesler, C., Brasser, F., Davi, L., Sadeghi, A.R.: Realswatt: Remote software-based attestation for embedded devices under realtime constraints. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS '21, p. 2890–2905. Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3460120.3484788
50. Tan, H., Hu, W., Jha, S.: A remote attestation protocol with trusted platform modules (tpms) in wireless sensor networks. Security and Communication Networks **8**(13), 2171–2188 (2015). https://doi.org/10.1002/sec.1162
51. TCG: Trusted computing group (2003). https://trustedcomputinggroup.org. Last visited: 2023–07-28
52. TCG: Tpm 2.0 authenticated countdown timer (act) command (2019). https://trustedcomputinggroup.org/wp-content/uploads/TCG_TPM_ACTCommand_v1r3_pubrev.pdf. Last visited: 2024–02-16
53. TCG: Tss 2.0 enhanced system api (esapi) specification (2021). https://trustedcomputinggroup.org/wp-content/uploads/TSS_ESAPI_v1p0_r14_pub10012021.pdf. Last visited: 2023–08-07
54. TCG: Dice attestation architecture (2023). https://trustedcomputinggroup.org/wp-content/uploads/DICE-Attestation-Architecture-Version-1.1-Revision-17_1August2023.pdf. Last visited: 2023–11-04
55. Usman, A.B., Asplund, M.: Remote attestation with software updates in embedded systems. In: 2024 IEEE Conference on Communications and Network Security (CNS), pp. 1–6 (2024). https://doi.org/10.1109/CNS62487.2024.10735526
56. Usman, A.B., Asplund, M.: Update at your own risk: Analysis and recommendations for update-related vulnerabilities. In: L. Nemec Zlatolas, K. Rannenberg, T. Welzer, J. Garcia-Alfaro (eds.) ICT Systems Security and Privacy Protection, pp. 97–110. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-92886-4_7
57. Usman, A.B., Cole, N., Asplund, M., Boeira, F., Vestlund, C.: Remote attestation assurance arguments for trusted execution environments. In: Proceedings of the 2023 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems, SaT-CPS '23, p. 33–42. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3579988.3585056
58. Wagner, P.G., Beyerer, J.: Quantifying trustworthiness in decentralized trusted applications. In: Proceedings of the 2022 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems, Sat-CPS '22, p. 67–76. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3510547.3517930
59. Wang, J., Wang, Y., Li, A., Xiao, Y., Zhang, R., Lou, W., Hou, Y.T., Zhang, N.: ARI: Attestation of real-time mission execution integrity. In: 32nd USENIX Security Symposium (USENIX Security 23), pp. 2761–2778. USENIX Association, Anaheim, CA (2023). https://www.usenix.org/conference/usenixsecurity23/presentation/wang-jinwen
60. Wilson, J., Asplund, M., Johansson, N., Boeira, F.: Provably secure communication protocols for remote attestation. In: Proceedings of the 19th International Conference on Availability, Reliability and Security, ARES '24. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3664476.3664485
61. Wu, Y., Wang, J., Wang, Y., Zhai, S., Li, Z., He, Y., Sun, K., Li, Q., Zhang, N.: Your firmware has arrived: A study of firmware update vulnerabilities (2024). https://www.usenix.org/conference/usenixsecurity24/presentation/wu-yuhao
62. Xu, M., Huber, M., Sun, Z., England, P., Peinado, M., Lee, S., Marochko, A., Mattoon, D., Spiger, R., Thom, S.: Dominance as a new trusted computing primitive for the internet of things. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 1415–1430 (2019). https://doi.org/10.1109/SP.2019.00084
63. Yadav, N., Ganapathy, V.: Whole-program control-flow path attestation. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS '23, p. 2680–2694. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3576915.3616687

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.